Agentic Coding

with Claude Code

Sankalp Sharma

University of Southern California ▪ April 27, 2026

High-Level Goals

- **Reducing startup costs for leveraging agentic coding tools**
- **Prompt Engineering**
- **Using agents effectively**: when to spawn one, when not to, what to ask.
- **Using the built-in features effectively**: plan mode, memory, hooks, skills, worktrees and so on

Three categories of AI coding tools

Chat

Claude.ai, ChatGPT

- You paste, it answers.
- No file access.
- Stateless.

IDE assistant

Cursor, Copilot

- Autocompletes as you type.
- Lives inside the editor.
- Line-level.

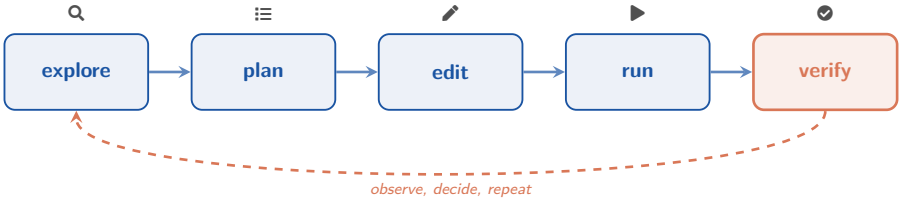
Agentic

Claude Code, Codex

- Reads files, plans, edits, runs, iterates.
- Runs in your terminal.
- Goal-level.

> « *Claude Code is a Unix utility, not an IDE.* »
>
> *Boris Cherny, Anthropic*

The agentic loop



- **Closed feedback.** Each tool call returns a result (a file, an error, a test failure) and that result becomes the next prompt
- **Goal-level, not step-level.** You say *"make the tests pass."* The model picks the steps.
- **Stops at the goal, not at the script.** The loop ends when the model decides the goal is met.

▸ hallucinations

Why economists should care

An empirical paper is a chain of expensive steps

Step	Before		With Claude Code
Clean a new admin dataset	2–5 RA-days	→	**a session**
Port `xtreg` to `reghdfe`	half a day	→	**10 minutes**
Build a publication-quality table	1–2 hours	→	**5 minutes**
Replicate someone else's paper	a week	→	**an evening**
Running hundreds of specifications	a weekend	→	**30 mins**
First-pass referee report on your own draft	*impossible*	→	**free**

▸ AEA compliance

Installation

macOS, Linux, or WSL

```
curl -fsSL https://claude.ai/install.sh |
    bash

# verify
claude --version
claude doctor

# launch
cd ~/my-project
claude
```

No paid plan?

Codex or the Cursor agent window work fine.

- Cursor is **free for students**.
- USC IT can grant access to **ChatGPT thinking mode + Codex**

Before you paste anything, what is safe to send?

Fine to send	Do not send
✔ Public data (Census, BLS, IPUMS)	✘ IRB-restricted human-subjects data
✔ Synthetic and anonymized panels	✘ Title 13 / 26 (FSRDC, IRS)
✔ Data schema	✘ Medical data
✔ Your code, drafts, slides	✘ PII microdata

Before you Claude, get on git

Version control is essential!

```
git init && gh repo create my-paper --private --source=.
echo "data/raw/"      >> .gitignore    # never commit raw data
echo "*.dta *.parquet *.log" >> .gitignore
echo ".env"           >> .gitignore    # never commit secrets
git add code/ paper/ CLAUDE.md && git commit -m "init"
```

Three habits that matter once Claude is editing your files:

- **One spec, one branch.** `git checkout -b feat/event-study`. Bad runs throw away cleanly.
- `git diff` **before approving.**
- **Worktrees for parallel sessions.** `git worktree add ../paper-did feat/did`, then cd in and run `claude`. Three Claudes, three modules

▸ git worktrees

Demo: arXiv econ.GN

One small project, end-to-end

Stage	What it covers
Scope	ToS check, plan-mode draft you approve before code runs.
Scrape	Paginated API, 3s rate limit, retries. 6,000 papers in $\sim$2 min.
Clean	LaTeX in abstracts, date parsing, cross-list explosion.
Explore	Submissions over time, category shares, Content analysis.
Write	Tables and figures piped into the deck.

Code: `sankalp/example/code/`. Outputs: `sankalp/example/output/`.

The five surfaces every config touches

Surface	What it does
`CLAUDE.md`	Project memory. Conventions, file layout, what to never do.
`settings.json`	Model, allow/deny rules, env vars, hook registration.
`.claude/agents/`	Subagents with separate context, custom system prompt.
`.claude/skills/`	Reusable workflows triggered by paths or arguments.
`.claude/hooks/`	Deterministic shell commands at lifecycle events.

CLAUDE.md

CLAUDE.md loads on every session. It is the single highest-leverage file in your project.

```
# CLAUDE.md (project root)
## Project: 1-line topic. Stack. Master entry point.
## Conventions: FE engine = reghdfe / fixest / pyfixest.
                Cluster SE level = district. Seed = 20260415.
                Tables = booktabs + esttab/modelsummary.
## Forbidden: pdflatex (use xelatex). pandas for >5GB.
               Plain TWFE for staggered designs.
## Replication: make all reproduces every table from data/raw/.
```

Two-tier setup:

- ~/.claude/CLAUDE.md (you, all projects): writing voice, banned phrases, default models.
- ./CLAUDE.md (project, committed): conventions, file layout, what never to touch.

Test for every line: *"Would removing this cause Claude to make a mistake?"* If not, cut.

path-scoped rules

Custom commands: /prompt turns dictation into a real prompt

A command lives at ~/.claude/commands/prompt.md. The command body is itself a prompt that runs when you type /prompt.

```
~/.claude/commands/prompt.md
~/.claude/commands/learn.md          # extract lessons from this session
~/.claude/commands/review.md         # review unpushed changes
~/.claude/commands/today.md          # session progress summary
~/.claude/commands/scrape.md         # build a scraper for a portal
~/.claude/commands/interview.md      # flesh out a spec via Q&A
```

What /prompt does: I dictate a sloppy half-sentence. The command (a) parses intent, (b) calibrates depth (light/standard/deep), (c) reformats into role + task + constraints + output format, (d) shows me the formatted version, (e) executes.

Net effect: the bottleneck moves from "what do I type" to "what do I want."

Skills: bundles of expertise the agent loads on demand

A skill is a folder with a `SKILL.md` (frontmatter + instructions) plus reference files.

Claude reads the frontmatter, decides if the skill is relevant, then pulls in the body.

```
~/.claude/skills/econometrics/          # OLS, IV, GMM, panel, treatment eval
~/.claude/skills/research-pipeline-qa/  # raw data to paper, end-to-end audits
~/.claude/skills/hpc/                   # Slurm scripts, USC CARC, modules
~/.claude/skills/scientific-visualization/  # journal-ready matplotlib/seaborn
```

Why skills beat one giant CLAUDE.md: Skills load only when triggered, so context stays cheap.

continuous learning

Hooks: deterministic shell commands that fire at lifecycle events

Hooks guarantee behavior (prompts only suggest it)

```
"PreToolUse":  pre-bash          # block rm -rf, git push --force, secrets
               pre-edit           # block edits to data/raw/, .env, IRB folders
               pre-write          # check no PII in new files
"PostToolUse": post-python-edit   # auto-format with ruff
               post-latex-edit    # try compiling, push errors back into
                   context
               post-tool-error    # capture errors for the [LEARN] memory
"SessionStart": auto-commit memory/ folder, sync configs across machines
"UserPromptSubmit": skill-router.py  # match prompt against regex, load skill
"Stop": claude-sync push         # back up the session
```

Agents: a fresh context window with a custom system prompt

Subagents live at `~/.claude/agents/<name>.md`.
Each one runs in isolation, returns only a summary. Main thread context stays clean.

```
~/.claude/agents/economics-researcher.md   # PhD-level Stata/R/Python/LaTeX
~/.claude/agents/code-reviewer.md          # convention compliance
~/.claude/agents/codebase-analyst.md       # explore, trace, impact analysis
~/.claude/agents/data-specialist.md        # SQL, ETL, BigQuery, profiling
~/.claude/agents/python-pro.md             # refactors, async, performance
~/.claude/agents/search-papers.md          # semantic search over Zotero
```

Opus as the advisor, Sonnet as the worker

Sonnet 4.6	The full session. 95% of edits, 95% of commands, 95% of routine debugging. Cheap, fast, plenty smart.
Opus 4.7	Pull in for: identification design, replication audits, "why does this estimator behave this way," stuck on a bug for 30+ minutes, reviewing your own writing.

The advisor pattern:

```
> /model opus
> Here is the spec, my data, my proposed identification.
  Critique it as if I were defending at AER. What is the
  weakest assumption? What would the editor flag?
> /model sonnet          # back to cheap mode for execution
```

Or set `/model opusplan` interactively: Opus during plan mode, Sonnet for execution.

model details

Monorepos

```
my-paper/
  brainstorm/     # half-formed ideas, the agent writes here
  plan/           # specs, written by you, edited by Claude
  code/           # the actual pipeline
  paper/          # main.tex, refs.bib, figures/, tables/
  source/         # PDFs, downloaded data, anything raw
  CLAUDE.md       # what to never touch, what conventions hold
```

Why this layout:

- One git repo, one place to point Claude.
- `brainstorm/` and `plan/` are version-controlled, so you can blame your past self.
- `source/` is gitignored; `code/` and `paper/` are committed.
- Path-scoped rules apply naturally: writing rules in `paper/`, code rules in `code/`.

Tips and Tricks

1. **Use `/plan`.** Separate "plan step" from "action step." This single habit fixes more bad runs than anything else
2. **Use natural language.** Claude parses dictation with "ums and ahs" just fine
3. **Use skills and plugins.** Skills are purpose-built instructions. Plugins are bundled skills. Both reduce "how do I word this" tax
4. **Use `-dangerously-skip-permissions`** (a.k.a. YOLO mode) once your hooks are solid

Questions

AEA Data Editor compliance

- As of April 2026, current policy is silent on AI authorship. Reproducibility requirements are unchanged. Policy is under active discussion, so check before submission.
- What gets deposited is your `.do` / `.R` / `.py`, not the model.
- Best practice: have Claude write `master.do` + the AEA-template README, then run Cunningham's Referee 2 cross-language replication before submission.
- Disclose AI use in a methods footnote: *"Code developed with Claude Code (Sonnet 4.6 / Opus 4.7)."*

back

Dealing with hallucinations

Two cases, very different consequences:

- **Prose and citations:** dangerous. BibTeX entries get invented. Verify every citation against your Zotero export.
- **Code:** Verify step is critical. Write unit tests!

Mitigations that work:

- Plan mode for any non-trivial task.
- Tests before code (Claude can write the tests too).
- `git diff` every change before approving.
- Referee 2 in a fresh session

back

Model Use

Model	Tier	Use for
Opus 4.x	Most expensive	Architecture, hard bugs, replication audits, identification design.
Sonnet 4.x	Default daily driver	95% of edits, debugging, routine sessions.
Haiku 4.x	Cheap and fast	Bulk mechanical work, file renames, log parsing.

- `/model opusplan`: Opus during plan, Sonnet for execution.
- Cache reads cost roughly 10% of input rate. Long sessions are cheaper than they look.
- Track via `/usage` and `npx ccusage daily`. Check Anthropic's pricing page for current per-token rates.

back

Path-scoped rules

Instead of one giant `CLAUDE.md`, split conventions by file type and let glob patterns do the routing:

```
~/.claude/rules/
  global.md        # applies always
  stata.md         # applies-to: **/*.do, **/*.ado
  python.md        # applies-to: **/*.py, **/*.ipynb
  r.md             # applies-to: **/*.R, **/*.Rmd
  data-work.md     # applies-to: **/data/**, **/build/**
  research.md      # applies-to: **/analysis/**, **/*.tex
  gis.md           # applies-to: **/*.shp, **/*.geojson
  writing.md       # applies-to: **/*.md, **/docs/**
```

Why this beats one big file:

- Stata-specific gotchas do not pollute Python sessions.
- Each file stays under the canonical 200-line limit.
- Sub-projects can override at the directory level.

back

Continuous learning

After every correction or surprising success, write a one-line memory. Future sessions read it on startup.

```
~/.claude/memory/
  MEMORY.md              # index, max 200 lines, one line per entry
  user_profile.md        # role, tools, knowledge
  feedback_*.md          # corrections you do not want to repeat
  project_*.md           # current state of in-flight work
```

The rule: after the second correction on the same issue, write a feedback memory. After the second time an unusual approach worked, write one too.

- *"PLFS weights need adjusted multipliers for round 7+."*
- *"For staggered DiD, default to Callaway-Sant'Anna."*
- *"Never use stargazer. Always modelsummary or esttab."*

Mistake rate falls fast once you have 20 entries. This is the closest thing to actual learning the agent does.

back

Git worktrees: three Claudes on three branches at once

```
# git creates a sibling checkout on a new branch
git worktree add ../paper-did feat/staggered-did
cd ../paper-did && claude    # fresh Claude session, isolated tree

# repeat for each parallel module
git worktree add ../paper-cleaning feat/clean-rounds

# list, then prune when done
git worktree list
git worktree remove ../paper-did
```

The pattern:

- Three terminal panes. Three branches. Three Claude sessions. Zero merge conflicts during the session because each lives in its own checkout.
- Scope by *module*, not by task. (`cleaning/`, `analysis/`, `paper/`.)
- Practical ceiling: 3–5 parallel sessions. Beyond that you cannot keep up with the diffs.
- Cost: a 3-agent hour costs about a single-agent day. Use deliberately.

back